(Multi-Agent) Reinforcement Learning: a primer, with applications in economics

MARL Algorithms and Challenges

Aldo Glielmo
Applied Research Team, Banca d'Italia

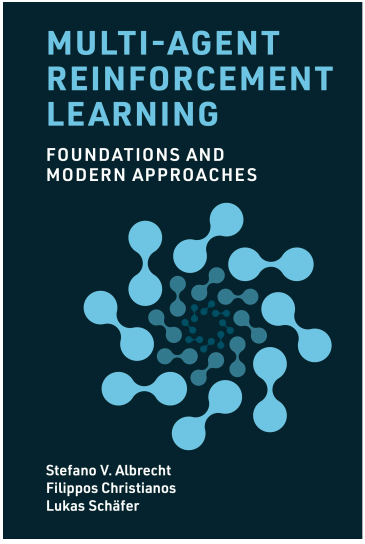
The MARL Book

A lot of material in this course is based on

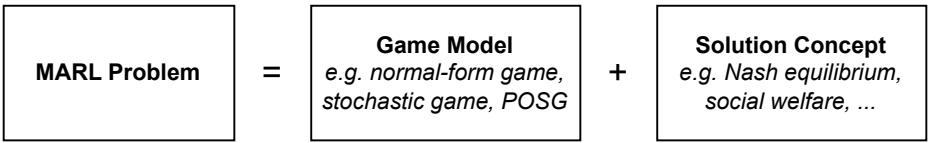
Multi-Agent Reinforcement Learning: Foundations and Modern Approaches

The book is freely available from:

www.marl-book.com



Recap: MARL Problem Definition



- Sequential decision process is defined formally as a Game Model
- Maximizing returns of one agent is no longer enough as a Solution Concept
- We must consider the **joint policy** of all agents
- There are different possible solution concepts

Recap: MARL Game Models and Solution Concepts

Game Models

- Normal-form games
- Stochastic games
- Partially observable stochastic games
- Game theory vs MARL assumptions

Solution Concepts

- Joint policy and expected return
- Equilibrium solution concepts

Today's Lecture

Single-agent reductions and challenges

- Learning framework for MARL
- Independent/central learning
- Self-play and mixed-play
- Challenges of MARL

  $\Rightarrow$ Short break (10 min, until around 11:20)

Foundational deep learning algorithms

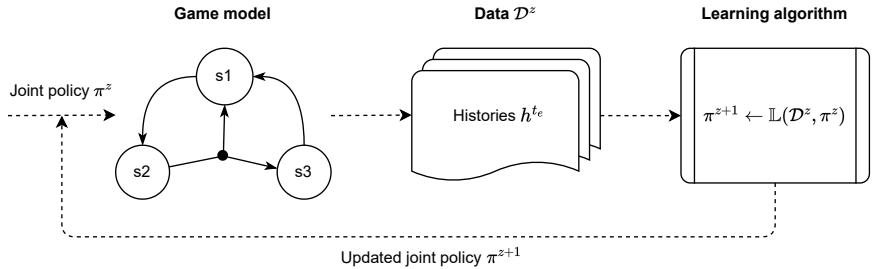
- Deep MARL (multi-agent policy gradient / centralised critics)
- Parameter and experience sharing

  $\Rightarrow$ Short break (15 min, until around 12:15)

Application in economics (Heterogeneous macro models) + discussion

MARL Learning Framework

MARL Learning Process



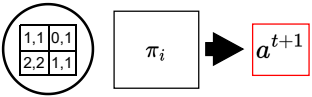
- The game model defines the learning environment
- Interaction data from joint policy π^z are collected as $\mathcal{D}^z = \{h^{t_e} \mid e = 1, \ldots, z\}, z \geq 0$
- A learning algorithm updates the joint policy as $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$
- The learning goal is a chosen solution concept, e.g. Nash equilibrium

MARL Learning Process - Continued

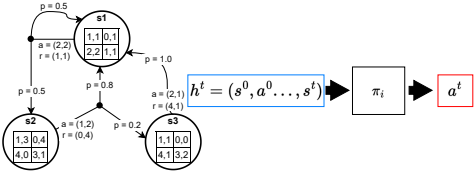
There are several additional nuances to the MARL learning process.

- The chosen game model will determine what the learnt joint policy conditions on:
  - Normal-form games: simple probability distribution across actions
  - Repeated normal-form games: condition on action histories $h^t = (a^0, ..., a^{t1})$
  - Stochastic games: condition on state-action histories $h^t = (s^0, a^0, s^1, a^1, ..., s^t)$
  - POSG: condition on independent observation histories $h_i^t = (o_i^0, ..., o_i^t)$
- Policies can however be adjusted as required e.g. in a stochastic game only condition on the last state-action
- The learning algorithm $\mathbb{L}$ can consists of multiple learning algorithms e.g. $\mathbb{L}_i \forall i \in I$
- Each $\mathbb{L}_i$ may use different parts of the data, or use independently collected data

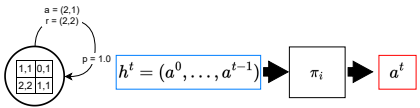
Inputs of Policies Depend on Game Model



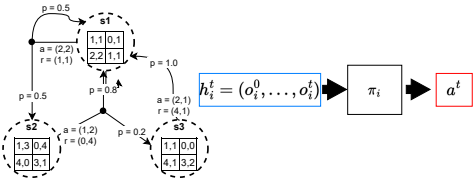
Normal-form games



Stochastic games



Repeated normal-form games



Partially observable stochastic games

Convergence

To evaluate the learning algorithm, we typically assess whether the learnt joint policy has **converged** to an optimal joint policy:

$$\lim_{z \to \infty} \pi^z = \pi^*$$

- Optimal joint policies may differ depending on the solution concept
- There may be many valid solutions $\Rightarrow$ e.g. multiple Nash equilibria
- In practice, we cannot collect infinite data $z \to \infty$

  $\Rightarrow$ Learning typically stops after a predefined 'budget' (e.g. training steps)

Single-Agent RL Reductions

The simplest way to apply RL algorithms in multi-agent settings is to reduce them to single-agent problems.

Central learning:

- Apply one single-agent RL algorithm to control all agents centrally

  $\Rightarrow$ A central policy is learned over the joint action space

Independent learning:

- Apply single-agent RL algorithms to each agent independently

  $\Rightarrow$ Agents do not explicitly consider or represent each other

Central Learning

Central Learning

Central learning: learn a single central policy π_c which receives observations of all agents and selects an action for each agent (i.e. joint action $(a_1, ..., a_n)$).

- Requires transforming the joint reward $(r_1, \ldots, r_n)$ into a single scalar reward r
- This can be easy in some settings, e.g. in games with common rewards $r = r_i$, but difficult in zero-sum or general-sum games
- May not scale well with the number of agents, as the joint action space may grow exponentially with the number of agents
- May also not be suitable in environments that require agents to act independently based on local observations

Central Q-Learning

Algorithm Central Q-learning

1: Initialize: $Q(s, a) = 0$ for all $s \in S$ and $a \in A = A_1 \times \ldots \times A_n$
2: Repeat for every episode:
3: **for** $t = 0, 1, 2, \ldots$ **do**
4: Observe current state s^t
5: With probability ϵ: choose random joint action $a^t \in A$
6: Otherwise: choose joint action $a^t \in \arg\max_a Q(s^t, a)$
7: Apply joint action a^t, observe rewards $r_1^t, \ldots, r_n^t$ and next state s^{t+1}
8: Transform $r_1^t, \ldots, r_n^t$ into scalar reward r^t
9: $Q(s^t, a^t) \leftarrow Q(s^t, a^t) + \alpha[r^t + \gamma \max_{a'} Q(s^{t+1}, a') - Q(s^t, a^t)]$

Independent Learning

Independent Learning

Independent learning: each agent i learns its policy π_i using only its local history of observations.

- From the perspective of each agent i, the environment transition function looks like this:

$$\mathcal{T}_i(s^{t+1}|s^t, a_i) \propto \sum_{a_{-i} \in A_{-i}} \mathcal{T}(s^{t+1}|s^t, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \pi_j(a_j|s^t)$$

- As each agent j's policies are updated, the action probabilities π_j change

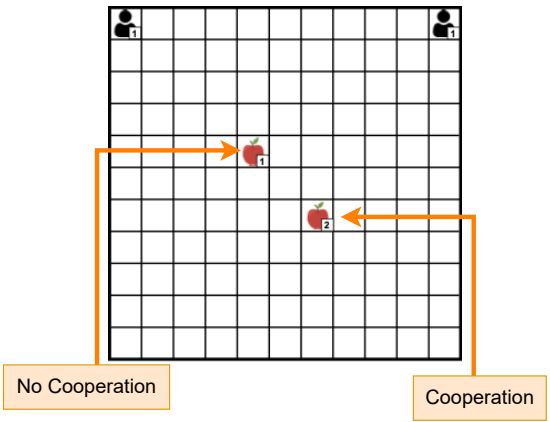
  $\Rightarrow$ From agent i's perspective, the transition function $\mathcal{T}_i$ appears non-stationary!

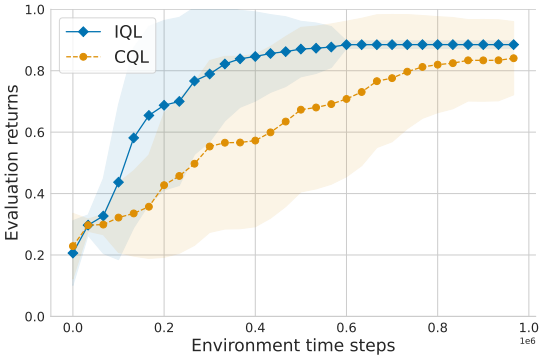
Independent Q-learning

Algorithm Independent Q-learning (IQL) for stochastic games

1: // *Algorithm controls agent i*
2: Initialize: $Q_i(s, a_i) = 0$ for all $s \in S$, $a_i \in A_i$
3: Repeat for every episode:
4: **for** $t = 0, 1, 2, \ldots$ **do**
5: Observe current state s^t
6: With probability ϵ: choose random action $a_i^t \in A_i$
7: Otherwise: choose action $a_i^t \in \arg\max_{a_i} Q_i(s^t, a_i)$
8: (meanwhile, other agents $j \neq i$ choose their actions a_j^t)
9: Observe own reward r_i^t and next state s^{t+1}
10: $Q_i(s^t, a_i^t) \leftarrow Q_i(s^t, a_i^t) + \alpha[r_i^t + \gamma \max_{a_i'} Q_i(s^{t+1}, a_i') - Q_i(s^t, a_i^t)]$

IQL and CQL in Level-Based Foraging





- IQL can learn more quickly, as CQL needs to explore $6^2 = 36$ actions in each state

Modes of Operation in MARL

Modes of operation in MARL:

Self-play:

- *Algorithm self-play:* all agents use the same learning algorithm (and parameters)
- *Policy self-play:* agent's policy is trained directly against itself

Mixed-play:

- Agents use different learning algorithms

Single-Agent Reductions – Interactive Visualisation

- Cooperative independent Q-learning: **link**
- Some things to check:
  - $\Rightarrow$ How does the results depend on the learning rate α?
  - $\Rightarrow$ What is the final policy of each agent? Does it converge to a cooperative policy?
  - $\Rightarrow$ How does the cooperative policy depend on the distribution of food items?

MARL Challenges

MARL Challenges

Singe-Agent RL Challenges

- Unknown environment dynamics
- Exploration-exploitation dilemma
- Non-stationarity from bootstrapping
- Temporal credit assignment

Multi-Agent RL Challenges

- Non-stationarity from multiple learning agents
- Equilibrium selection
- Multi-agent credit assignment
- Scaling to many agents

Non-Stationarity

A stochastic process $X^t{}_{t\in\mathbb{N}^0}$ is stationary if:

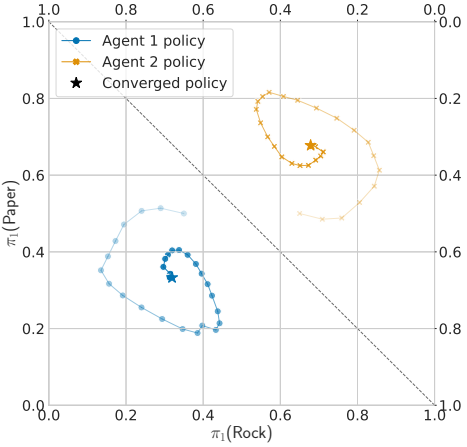
- The probability distribution of $X^{t+\tau}$ does not depend on $\tau \in \mathbb{N}^0$, where t and $t+\tau$ are time indices
- This means that the dynamics of the process do not change over time

Consider: X^t samples the state s^t at each time step t:

- In a MDP, X^t is completely defined by the state transition function $\mathcal{T}(s^t|s^{t-1}, a^{t-1})$ and the agent's policy π which selects an action $a \sim \pi(.|s)$
- If π does not change, then this process is stationary (i.e. independent of t) because s^t depends only on s^{t-1}, a^{t-1} and a^{t-1} depends only on s^{t-1} via $\pi(.|s^{t-1})$
- However, in RL the policy does change over time through the learning $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$, which leads to **non-stationarity** in X^t

Non-Stationarity in Multi-Agent Settings

In MARL, non-stationarity is exacerbated by multiple agents changing their policies!



- $\pi^{z+1} = \mathbb{L}(\mathcal{D}^z, \pi^z)$ updates an *entire* joint policy $\pi^z = (\pi_1^z, ..., \pi_n^z)$
- Environment is non-stationary from each agent's perspective
- Can cause cyclic learning dynamics where agents co-adapt to each other's changing policies

Equilibrium Selection

Equilibrium selection: a game may have multiple equilibria, which can yield different expected returns to the agents.

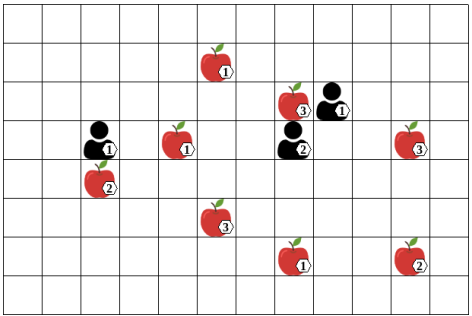
- **Example:** Stag Hunt matrix game
- Two hunters choose: cooperate to hunt stag (S) or go solo for hare (H)
- Nash equilibria: reward-dominant (S,S) maximizes reward, risk-dominant (H,H) minimizes risk
- (S,S) requires trust; (H,H) offers a safe, lower reward

	S	H
S	4,4	0,3
H	3,0	2,2

- Indep. Q-learning may bias towards (H,H) due to initial action uncertainty
- Early random actions can reinforce (H,H) since deviating from H is penalized if the other agent chooses H

Multi-Agent Credit Assignment

Multi-agent credit assignment: which agent's actions contributed to received rewards?



- At time step t all agents perform *collect*, each receiving reward $+1$
- Whose actions led to the reward?
- The agent on the left did not contribute
- For a learning agent that only observes s^t, a^t, r^t, s^{t+1}, disentangling the action contributions is difficult

Joint Actions for Addressing Multi-Agent Credit Assignment

Joint actions can help disentangle agent contributions. Consider the RPS game:

	R	P	S
R	0,0	-1,1	1,-1
P	1,-1	0,0	-1,1
S	-1,1	1,-1	0,0

1. Agents choose actions $(a_1, a_2) = (R, S)$ $\Rightarrow$ agent 1 receives reward $+1$
2. Then agents choose $(a_1, a_2) = (R, P)$ $\Rightarrow$ agent 1 receives reward -1
3. Action value $Q(a_1)$ does not model agent 2, value for action R appears to be 0
4. Joint action value model $Q_1(a_1, a_2)$ can represent the effect of agent 2: $Q_1(R, S) = +1$ and $Q_1(R, P) = -1$

Scaling to Many Agents

Problem

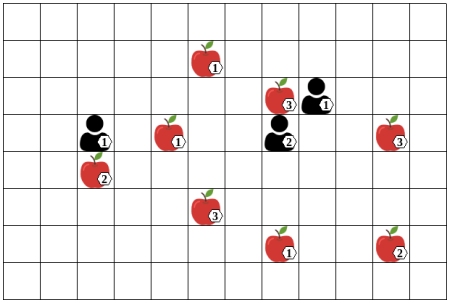
- **Joint action space** can grow **exponentially** with number of agents:

$$|A| = |A_1| \cdot ... \cdot |A_n|$$

- If agents have associated features in s (e.g. agent position) then $|S|$ also increases exponentially

  $\Rightarrow$ How to scale efficiently to many agents?

Scaling to Many Agents



Changing number of agents from 3 to 5 increases the number of joint actions from 216 to 7776!

Training and Execution Modes

Training and Execution Modes

We often distinguish between training and execution modes in MARL:

- **Training**: what information is available to agents during learning?
- **Execution**: what information is available to agents for action selection?

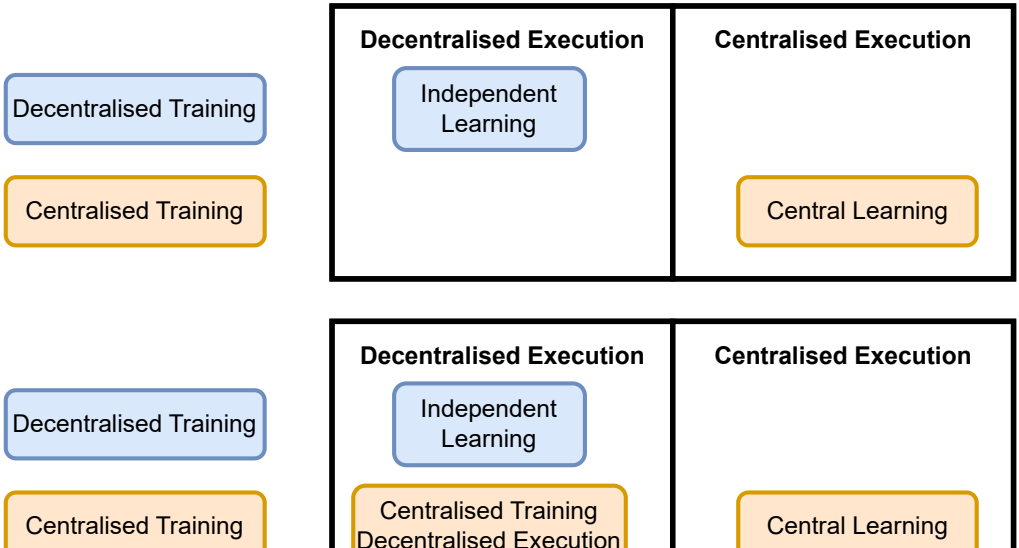
Reminder

We have already seen two single-agent reductions of the MARL problem:

- **Independent learning**: each agent learns its policy independently
  $\rightarrow$ decentralized training and decentralized execution
- **Central learning**: learn single policy over the joint action space
  $\rightarrow$ centralized training and centralized execution

But we can also have centralised training with decentralised execution (CTDE)!

Training and Execution Modes



Independent Learning with Deep Reinforcement Learning

Reminder

In the independent learning framework, each agent i learns its policy π_i using only its local history of observations, treating the effects of other agents' actions as part of the environment.

- From the perspective of the individual agent, the environment transition function looks like this:

$$\mathcal{T}_i(s^{t+1}|s^t, a_i) \propto \sum_{a_{-i} \in A_{-i}} \mathcal{T}(s^{t+1}|s^t, \langle a_i, a_{-i} \rangle) \prod_{j \neq i} \pi_j(a_j|s^t)$$

How about we do this with deep RL? We have already seen several single-agent deep RL algorithms: DQNDQN, REINFORCE, A2CA2C, PPO, etc.

Independent Deep Q-Networks

Algorithm Independent deep Q-networks

1: Initialize n value networks with random parameters $\theta_1, \ldots, \theta_n$
2: Initialize n target networks with parameters $\bar{\theta}_1 = \theta_1, \ldots, \bar{\theta}_n = \theta_n$
3: Initialize a replay buffer for each agent $D_1, D_2, \ldots, D_n$
4: **for** time step $t = 0, 1, 2, \ldots$ **do**
5: Collect current observations $o_1^t, \ldots, o_n^t$
6: **for** agent $i = 1, \ldots, n$ **do**
7: With probability ϵ: choose random action a_i^t
8: Otherwise: choose $a_i^t \in \arg\max_{a_i} Q(h_i^t, a_i; \theta_i)$
9: Apply actions $(a_1^t, \ldots, a_n^t)$; collect rewards $r_1^t, \ldots, r_n^t$ and next observations $o_1^{t+1}, \ldots, o_n^{t+1}$
10: **for** agent $i = 1, \ldots, n$ **do**
11: Store transition $(h_i^t, a_i^t, r_i^t, h_i^{t+1})$ in replay buffers D_i
12: Sample random mini-batch of B transitions $(h_i^k, a_i^k, r_i^k, h_i^{k+1})$ from D_i
13: **if** s^{k+1} is terminal **then**
14: Targets $y_i^k \leftarrow r_i^k$
15: **else**
16: Targets $y_i^k \leftarrow r_i^k + \gamma \max_{a_i' \in A_i} Q(h_i^{k+1}, a_i'; \bar{\theta}_i)$
17: Loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{B} \sum_{k=1}^{B} \left(y_i^k - Q(h_i^k, a_i^k; \theta_i) \right)^2$
18: Update parameters θ_i by minimizing the loss $\mathcal{L}(\theta_i)$
19: In a set interval, update target network parameters $\bar{\theta}_i$

- Almost identical to single-agent DQN but with n agents!
- Replay buffer contains off-policy experiences due to changing policies
- In MARL, the policies of all agents are changing $\rightarrow$ training can be unstable

Independent Advantage Actor-Critic

Algorithm Independent A2C with synchronous environments

Initialize n actor networks with random parameters $\phi_1, \ldots, \phi_n$
Initialize n critic networks with random parameters $\theta_1, \ldots, \theta_n$
Initialize K parallel environments
for time step $t = 0 \ldots$ **do**
    Batch of observations for each agent and environment: $\begin{bmatrix} o_1^{t,1} \ldots o_1^{t,K} \\ \ddots \\ o_n^{t,1} \ldots o_n^{t,K} \end{bmatrix}$
    Sample actions $\begin{bmatrix} a_1^{t,1} \ldots a_1^{t,K} \\ \ddots \\ a_n^{t,1} \ldots a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \ldots, \pi(\cdot \mid h_n^t; \phi_n)$
    Apply actions; collect rewards $\begin{bmatrix} r_1^{t,1} \ldots r_1^{t,K} \\ \ddots \\ r_n^{t,1} \ldots r_n^{t,K} \end{bmatrix}$ and observations $\begin{bmatrix} o_1^{t+1,1} \ldots o_1^{t+1,K} \\ \ddots \\ o_n^{t+1,1} \ldots o_n^{t+1,K} \end{bmatrix}$
    for agent $i = 1, \ldots, n$ **do**
        if $s^{t+1,k}$ is terminal **then**
            Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}; \theta_i)$
            Critic target $y_i^{t,k} \leftarrow r_i^{t,k}$
        else
            Advantage $Adv(h_i^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i) - V(h_i^{t,k}; \theta_i)$
            Critic target $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}; \theta_i)$
        Actor loss $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^{K} Adv(h_i^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$
        Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^{K} \left(y_i^{t,k} - V(h_i^{t,k}; \theta_i) \right)^2$
        Update parameters ϕ_i by minimizing the actor loss $\mathcal{L}(\phi_i)$
        Update parameters θ_i by minimizing the critic loss $\mathcal{L}(\theta_i)$

- Almost identical to single-agent A2C
- Similar adaptation can be done for independent REINFORCE and independent PPO

Challenges of Multi-Agent Reinforcement Learning

Reminder

MARL algorithms suffer from multi-agent specific challenges:

- **Non-stationarity**: exacerbated due to changing policies of all agents
- **Equilibrium selection**: how to converge to a stable equilibrium?
- **Multi-agent credit assignment**: how to attribute rewards to agents' actions?
- **Scaling to many agents**: how to efficiently scale to large numbers of agents?

Centralised training with decentralised execution (CTDE) can help address some of these challenges.

Multi-Agent Policy Gradient Algorithms

The Policy-Gradient Theorem

Reminder

Follow this gradient to optimise the parameters ϕ of the policy π to maximise the expected return:

$$\begin{aligned}\nabla_\phi J(\phi) &\propto \sum_{s \in S} \Pr(s \mid \pi) \sum_{a \in A} Q^\pi(s, a) \nabla_\phi \pi(a \mid s; \phi) \\ &= \mathbb{E}_{s \sim \Pr(\cdot \mid \pi), a \sim \pi(\cdot \mid s; \phi)} [Q^\pi(s, a) \nabla_\phi \log \pi(a \mid s; \phi)]\end{aligned}$$

Does this also hold for MARL? Yes, with minor modifications!

The Multi-Agent Policy-Gradient Theorem

Solution

In MARL, the expected returns of agent i under its policy π_i depends on the policies of all other agents $\pi_{-i} \rightarrow$ the multi-agent policy gradient theorem defines an expectation over the policies of **all** agents:

$$\nabla_{\phi_i} J(\phi_i) \propto \mathbb{E}_{\hat{h}\sim \Pr(\hat{h}|\pi), a_i\sim\pi_i, a_{-i}\sim\pi_{-i}} \Big[Q_i^{\pi}(\hat{h}, \langle a_i, a_{-i}\rangle) \nabla_{\phi_i} \log \pi_i(a_i \mid h_i = \sigma_i(\hat{h}); \phi_i) \Big]$$

Derive policy update rules by finding estimators for expected return $Q_i^{\pi}(\hat{h}, \langle a_i, a_{-i}\rangle)$

We have already seen independent A2C that estimates $Adv(h_i, a_i) \approx Q_i^{\pi}(\hat{h}, \langle a_i, a_{-i}\rangle)$

But can we do better? Perhaps by leveraging more information?

Centralized Critics

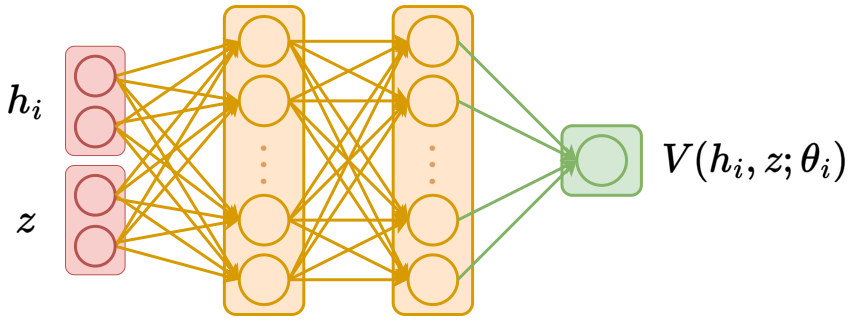
Note

In actor-critic algorithms, only the policy/actor is used during execution and the critic is used only during training $\rightarrow$ the critic can be conditioned on centralised information z without compromising decentralised execution.

This might include:

- Global state s
- Joint action a
- Joint observation history h
- ...

Centralized Critics



Now we can integrate centralized critics into multi-agent policy gradient algorithms.

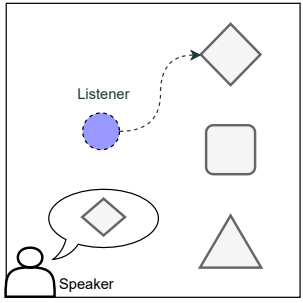
Centralized Advantage Actor-Critic

Algorithm Centralized A2C with synchronous environments

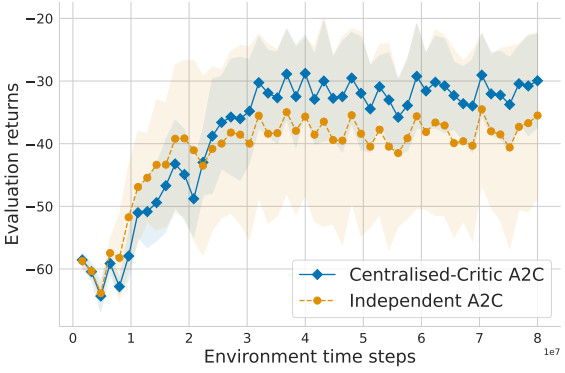
Initialize n actor networks with random parameters $\phi_1, \ldots, \phi_n$
Initialize n critic networks with random parameters $\theta_1, \ldots, \theta_n$
Initialize K parallel environments
for time step $t = 0 \ldots$ **do**
  Batch of observations for each agent and environment: $\begin{bmatrix} o_1^{t,1} \ldots o_1^{t,K} \\ \ddots \\ o_n^{t,1} \ldots o_n^{t,K} \end{bmatrix}$
  Batch of centralized information for each environment: $\left[z^{t,1} \ldots z^{t,K}\right]$
  Sample actions $\begin{bmatrix} a_1^{t,1} \ldots a_1^{t,K} \\ \ddots \\ a_n^{t,1} \ldots a_n^{t,K} \end{bmatrix} \sim \pi(\cdot \mid h_1^t; \phi_1), \ldots, \pi(\cdot \mid h_n^t; \phi_n)$
  Apply actions; collect rewards $\begin{bmatrix} r_1^{t,1} \ldots r_1^{t,K} \\ \ddots \\ r_n^{t,1} \ldots r_n^{t,K} \end{bmatrix}$, observations $\begin{bmatrix} o_1^{t+1,1} \ldots o_1^{t+1,K} \\ \ddots \\ o_n^{t+1,1} \ldots o_n^{t+1,K} \end{bmatrix}$, and centralized information $\left[z^{t+1,1} \ldots z^{t+1,K}\right]$
  for agent $i = 1, \ldots, n$ **do**
    if $s^{t+1,k}$ is terminal **then**
      $Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} - V(h_i^{t,k}, z^{t,k}; \theta_i)$
      Critic target $y_i^{t,k} \leftarrow r_i^{t,k}$
    else
      $Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}, z^{t+1,k}; \theta_i) - V(h_i^{t,k}, z^{t,k}; \theta_i)$
      Critic target $y_i^{t,k} \leftarrow r_i^{t,k} + \gamma V(h_i^{t+1,k}, z^{t+1,k}; \theta_i)$
    Actor loss $\mathcal{L}(\phi_i) \leftarrow \frac{1}{K} \sum_{k=1}^{K} Adv(h_i^{t,k}, z^{t,k}, a_i^{t,k}) \log \pi(a_i^{t,k} \mid h_i^{t,k}; \phi_i)$
    Critic loss $\mathcal{L}(\theta_i) \leftarrow \frac{1}{K} \sum_{k=1}^{K} \left(y_i^{t,k} - V(h_i^{t,k}, z^{t,k}; \theta_i)\right)^2$
    Update parameters ϕ_i by minimizing the actor loss $\mathcal{L}(\phi_i)$
    Update parameters θ_i by minimizing the critic loss $\mathcal{L}(\theta_i)$

- Simple extension of independent A2C.
- Centralized information z is added to the critic input.

Centralized Critics in Action



(a) Speaker-listener game

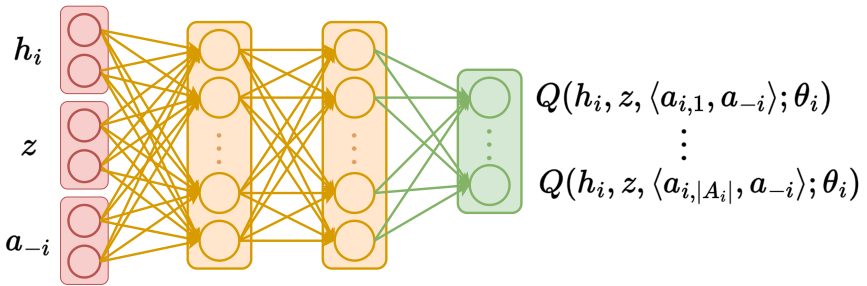


(b) Training curves

Agents with centralized critics converge to higher returns than agents with independent critics in the partially observable speaker-listener game.

Centralized Action-Value Critics

Similarly, we can learn an action-value function that receives additional centralized information z.



But what for? The centralized action-value critic can reason about the joint action space!

Counterfactual Multi-Agent Policy Gradient

For example, we can compute a counterfactual advantage for agent i

$$Adv_i(h_i, z, a) = Q(h_i, z, a; \theta) - \underbrace{\sum_{a'_i \in A_i} \pi(a'_i \mid h_i; \phi_i) Q(h_i, z, \langle a'_i, a_{-i} \rangle; \theta)}_{\text{counterfactual baseline}}$$

with the following components:

- $Q(h_i, z, a; \theta)$: expected returns when applying joint action $a \rightarrow$ agent i applies action a_i and all other agents apply actions a_{-i}
- Counterfactual baseline: expected returns when agent i samples its action a'_i from its policy $\pi(\cdot; \phi_i)$ and all other agents apply their actual actions a_{-i}

Identify contribution of agent i's action a_i to received rewards $\Rightarrow$ help to address the credit assignment problem in common-reward games

Centralized Critic Algorithms

Commonly seen centralized critic algorithms:

- MAA2C: A2C agent with a centralized critic
- COMA: MAA2C with Counterfactual Multi-Agent Policy Gradients
- MADDPG: DDPG agent with a centralized Q
- MATRPO: TRPO agent with a centralized critic
- MAPPO: PPO agent with a centralized critic
- HATRPO: Sequentially updating critic of MATRPO agents
- HAPPO: Sequentially updating critic of MAPPO agents

MARL – Interactive Demonstration

- Colab notebook: **link**
- Some things to check:

  $\Rightarrow$ How do the losses evolve during training?

  $\Rightarrow$ How does the reward evolve during training?

  $\Rightarrow$ Do you understand what happens during the training process?

  $\Rightarrow$ Do you understand the different hyperparameters and their effect on the learning process?

Parameter and Experience Sharing

Parameter and Experience Sharing – Motivation

Problem

Training agents with MARL is difficult for environments with many agents due to the increased number of parameters to train $\Rightarrow$ unstable or slow training!

Solution

Two approaches to improve the efficiency of training many agents:

- **Parameter sharing**: agents share their network parameters with each other
- **Experience sharing**: agents share experiences with each other

Environments with Homogeneous Agents

- Parameter and experience sharing assume that agents are trying to learn similar or identical policies $\rightarrow$ not applicable to all environments
- We call agents in such environments homogeneous
  - Strongly homogeneous agents: All agents have the same optimal policy, i.e.

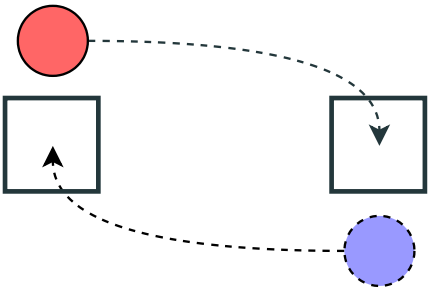
$$\pi_1^* = \ldots = \pi_n^*$$

  - Weakly homogeneous agents: Agents can be permuted and their expected returns remain the same under the permutation $\sigma : I \mapsto I$:

$$U_i(\pi) = U_{\sigma(i)}\left(\langle \pi_{\sigma(1)}, \pi_{\sigma(2)}, \ldots, \pi_{\sigma(n)} \rangle\right), \ \forall i \in I$$

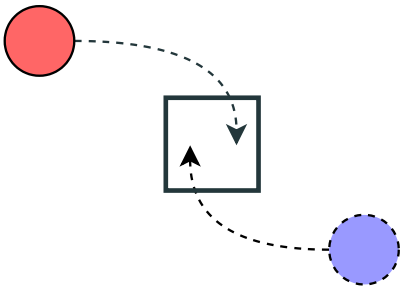
Environments with Homogeneous Agents – Examples

Weakly homogeneous agents:



Agents need to learn similar policies.

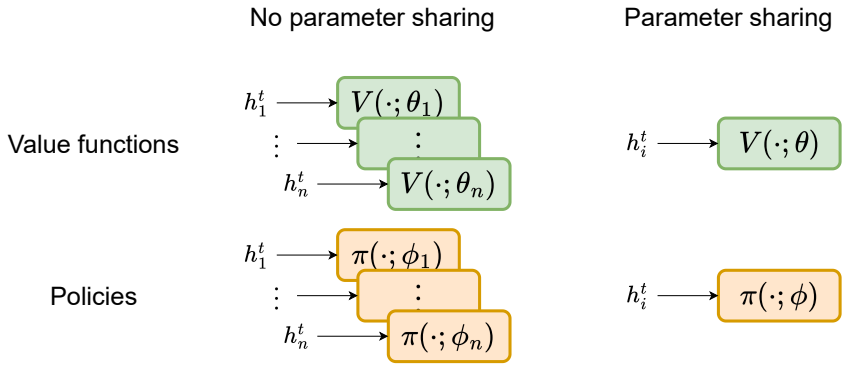
Strongly homogeneous agents:



Agents need to learn identical policies.

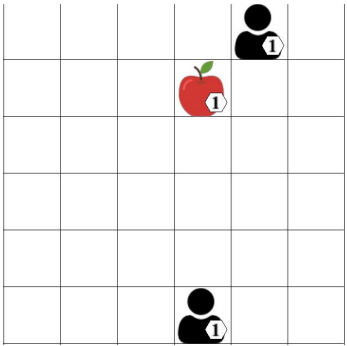
Parameter Sharing

Sharing network parameters across agents is common practice to make MARL training more efficient. Share parameters across value functions, policies, or both.

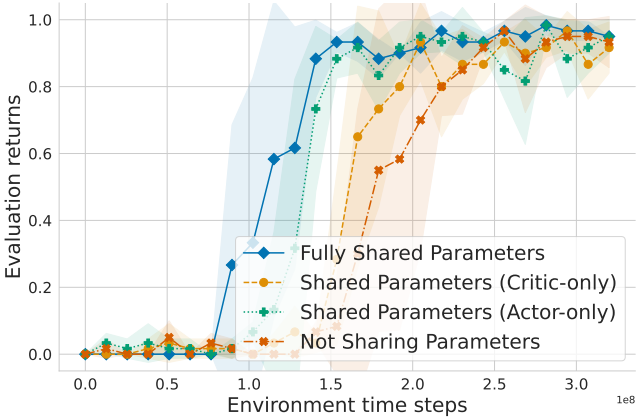


Parameter sharing has two primary benefits:

Parameter Sharing in LBF



(a) Environment



(b) Learning curve

Experience Sharing for Weakly Homogeneous Agents

Under experience sharing, agents still learn separate policies (and value functions) but share their trajectories to allow training on the collective data.

Assumption: Agents are (at least) weakly homogeneous, i.e. they need to learn similar policies.

Note

The experiences of agent j is **off-policy** data for agent $i \rightarrow$ experience sharing needs to use off-policy MARL algorithms or correct for the differences in data distributions.

Deep Q-Networks with Shared Experience Replay

We can extend IDQN with experience sharing by following the steps below:

- Collect the experience of all agents in a shared replay buffer $\mathcal{D}_{\text{shared}}$
- Each agent samples from $\mathcal{D}_{\text{shared}}$ to update its value function
- Each agent optimises its value function using the typical IDQN loss

Note

DQN is an off-policy algorithm so it is theoretically sound to use the experience of other agents that have different policies.

Shared Experience Actor-Critic (SEAC)

We can also extend independent actor-critic algorithms (e.g. IA2C and IPPO) with experience sharing by optimising the loss over the data of all agents.

SEAC policy loss (based on IA2C) with importance sampling (IS) weight correction:

policy loss on own data

$$\mathcal{L}(\phi_i) = -\left(r_i^t + \gamma V(h_i^{t+1}; \theta_i) - V(h_i^t; \theta_i)\right) \log \pi(a_i^t \mid h_i^t; \phi_i)$$
$$- \lambda \sum_{k \neq i} \frac{\pi(a_k^t \mid h_k^t; \phi_i)}{\pi(a_k^t \mid h_k^t; \phi_k)} \left(r_k^t + \gamma V(h_k^{t+1}; \theta_i) - V(h_k^t; \theta_i)\right) \log \pi(a_k^t \mid h_k^t; \phi_i)$$

IS correction

policy loss on data of agent k

Hyperparameter λ determines weighting for the loss over the experience of other agents. The same IS weight correction can be applied to the critic loss.

MARL Results: AlphaStar (StarCraft II)

AlphaStar — StarCraft II

- First AI to reach **Grandmaster level**
- **League training**: main agents + exploiters compete simultaneously
- Handles partial observability, long horizons, large action spaces
- Policy conditioned on a statistic encoding opponent's style
- DeepMind, *Nature* 2019 **video**

Vinyals et al., *Nature* 575, 350–354 (2019)

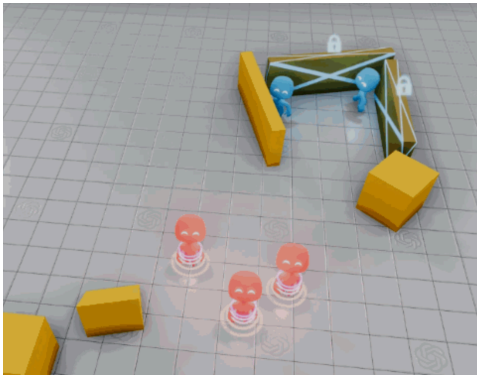
MARL Results: OpenAI Five (Dota 2)

OpenAI Five — Dota 2

- 5v5 cooperative team defeats **world champions** (OG, 2019)
- Self-play at scale: ∼180 years of gameplay per day
- No explicit communication — coordination via **shared policy + LSTM**
- Long-horizon cooperation: 45-min games, 20k+ timesteps
- OpenAI, 2019 **video**

Berner et al., arXiv:1912.06680 (2019)

MARL Results: OpenAI (Hide & Seek)



OpenAI Five — Hide & Seek

- **6 phases** of emergent behavior from a simple reward
- Agents learn to use and abuse tools (boxes, ramps) spontaneously
- Co-evolutionary arms race: hiders & seekers **mutually drive complexity**
- Emergent behaviors act as **unsupervised curriculum**
- OpenAI, ICLR 2020 **video**

Baker et al., ICLR 2020

Summary

We covered:

- MARL learning process
- Independent and central learning
- Self-play and mixed-play
- Challenges of MARL
- Centralised and decentralised training and execution
- Independent learning with deep learning
- Multi-agent policy gradient theorem
- Centralized critics and their benefits
- Parameter and experience sharing for homogeneous agents

Interactive Coding and Learning Session

MARL – Textbook References

References

- *Multi-Agent Reinforcement Learning: Foundations and Modern Approaches*. See chapters 9-11. A possible future "bible" for multi-agent RL. **link**
- *Multiagent Systems: Algorithmic, Game-Theoretic, and Logical Foundations*. See chapters 7. A classic book for MAS. **link**

Computational Resources

- **marl-book code**

MARL – Foundational Papers I

- IQL/CQL: *Multi-Agent Reinforcement Learning: Independent vs. Cooperative Agents*, Tan, *ICML*, 1993. **link**
- MAPPO: *The Surprising Effectiveness of PPO in Cooperative Multi-Agent Games*, Yu et al., *NeurIPS*, 2022. **link**
- MADDPG: *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*, Lowe et al., *NeurIPS*, 2017. **link**
- SEAC: *Shared Experience Actor-Critic for Multi-Agent Reinforcement Learning*, Christianos et al., *NeurIPS*, 2020. **link**
- *Scaling Multi-Agent Reinforcement Learning with Selective Parameter Sharing*, Christianos et al., *ICML*, 2021. **link**
- *Benchmarking Multi-Agent Deep Reinforcement Learning Algorithms in Cooperative Tasks*, Papoudakis et al., *NeurIPS*, 2021. **link**
- *Awesome MARL GitHub paper collection* **link**

MARL – Foundational Papers II

Self-Play and Tree Search

- MCTS: *A Survey of Monte Carlo Tree Search Methods*, Browne et al., *IEEE TCIAIG*, 2012. **link**
- AlphaZero: *A General Reinforcement Learning Algorithm that Masters Chess, Shogi, and Go Through Self-Play*, Silver et al., *Science*, 2018. **link**
- AlphaStar: *Grandmaster Level in StarCraft II Using Multi-Agent Reinforcement Learning*, Vinyals et al., *Nature*, 2019. **link**
- PSRO: *A Unified Game-Theoretic Approach to Multiagent Reinforcement Learning*, Lanctot et al., *NeurIPS*, 2017. **link**
- *Open-Ended Learning Leads to Generally Capable Agents*, Open-Ended Learning Team, *arXiv*, 2021. **link**

MARL – Computational Resources (Python / Jax)

Environments

- **PettingZoo** – multi-agent environment interface and suite
- **VMAS** – a GPU-accelerated multi-agent environment simulator
- **JaxMARL** – JAX-accelerated environments (and algorithms)
- **OpenSpiel** – games and environments, with some classical solvers

Algorithms

- **BenchMARL** – standardised benchmarking of MARL algorithms
- **Mava** – JAX-accelerated MARL algorithm implementations

References

- *Heterogeneous RBCs via Deep Multi-Agent Reinforcement Learning*, Gabriele et al., *AAMAS*, 2026. **link**

Computational Resources

- **Python code**